

1.4 데이터 핸들링 - 판다스(Pandas)

목차

1. Pandas 시작 - 파일을 DataFrame으로 로딩, 기본 API

1. Pandas 개요와 핵심 자료구조
2. `read_csv()`로 파일 로딩
3. `head()` / `shape`
4. `info()` - 데이터 요약 정보
5. `describe()` - 기술 통계량
6. `value_counts()` - 값 분포 확인
7. Series와 DataFrame의 차이
8. `dropna` 옵션과 결측치 카운트

2. DataFrame과 리스트·딕셔너리·ndarray 상호 변환

1. 리스트/ndarray → DataFrame
2. 2차원 데이터 → DataFrame
3. 딕셔너리 → DataFrame
4. DataFrame → ndarray/리스트/딕셔너리

3. DataFrame의 컬럼 데이터 세트 생성과 수정

4. DataFrame 데이터 삭제 - `drop()`

1. 컬럼 삭제(`axis=1`)
2. `inplace` 옵션
3. 로우 삭제(`axis=0`)

5. Index 객체

1. Index 객체의 추출과 특성
2. Series 연산에서 Index의 역할
3. `reset_index()`

6. 데이터 셀렉션 및 필터링

1. `[]` 연산자
2. `iloc[]` - 위치 기반 인덱싱
3. `loc[]` - 명칭 기반 인덱싱
4. 불린 인덱싱

7. 정렬, Aggregation 함수, GroupBy

1. `sort_values()`
2. Aggregation 함수
3. `groupby()` 활용

8. 결손 데이터 처리 - `isna()`와 `fillna()`

9. apply lambda 식으로 데이터 가공

10. 핵심 요약 및 머신러닝 활용 포인트

1. Pandas 시작 - 파일을 DataFrame으로 로딩, 기본 API

1.1 Pandas 개요와 핵심 자료구조

판다스(Pandas)는 파이썬에서 데이터 처리를 위해 존재하는 가장 인기 있는 라이브러리입니다. 일반적으로 대부분의 데이터 세트는 행(row)과 열(column)로 구성된 2차원 데이터이며, 판다스는 이러한 **2차원 데이터를 효율적으로 가공·처리**할 수 있는 다양한 기능을 제공합니다. NumPy가 저수준 API 중심으로 사용이 다소 어려운 반면, 판다스는 훨씬 직관적이고 유연하게 데이터를 다룰 수 있게 해 줍니다.

자료구조	설명
DataFrame	컬럼(열)이 여러 개인 2차원 데이터 세트 . 판다스의 핵심 객체이며 엑셀 시트나 데이터베이스 테이블과 유사합니다.
Series	컬럼이 하나뿐인 1차원 데이터 세트 . DataFrame에서 컬럼 하나를 뽑으면 Series가 됩니다.
Index	DataFrame/Series의 레코드를 고유하게 식별하는 키 값 . RDBMS의 Primary Key와 유사한 역할을 합니다.

■ 실습 데이터 소개 - 타이타닉 데이터 세트

본 자료는 캐글(Kaggle)의 대표적인 입문 데이터인 `titanic_train.csv`를 사용합니다. 891명의 탑승객에 대한 12개 컬럼(생존 여부, 객실 등급, 이름, 성별, 나이, 요금 등)으로 구성되어 있으며, 결측치가 적절히 포함되어 있어 데이터 전처리 학습에 이상적입니다.

1.2 read_csv()로 파일 로딩

```
import pandas as pd
```

줄	코드	상세 설명
1	<code>import pandas as pd</code>	pandas 모듈을 <code>pd</code> 라는 별칭으로 임포트합니다. NumPy의 <code>np</code> 와 마찬가지로 전 세계적으로 통용되는 관례이므로 반드시 이 형태를 사용합니다.

```
titanic_df = pd.read_csv('titanic_train.csv')
print('titanic 변수 type:', type(titanic_df))
titanic_df
```

줄	코드	상세 설명
1	<code>titanic_df = pd.read_csv('titanic_train.csv')</code>	<code>read_csv()</code> 는 CSV 파일을 읽어 DataFrame으로 변환 합니다. 첫 번째 인자는 파일 경로(filepath_or_buffer)입니다. 별도로 지정하지 않으면 파일의 첫 번째 줄이 컬럼명 으로 자동 인식되고, 인덱스는 0부터 시작하는 <code>RangeIndex</code> 가 자동 부여됩니다. 구분자가 콤마가 아니면 <code>sep='\t'</code> 처럼 지정합니다.
2	<code>print('titanic 변수 type:', type(titanic_df))</code>	결과는 <code><class 'pandas.core.frame.DataFrame'></code> . 로딩된 객체가 DataFrame임을 확인합니다.
3	<code>titanic_df</code>	주피터 노트북에서 변수명만 입력하면 DataFrame이 표 형태로 출력 됩니다. 행이 많으면 앞뒤 일부만 표시되고 중간은 <code>...</code> 으로 생략됩니다.

▶ 실행 결과

```
titanic 변수 type: <class 'pandas.core.frame.DataFrame'>
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen H...	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. L...	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jac...	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William...	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
887	888	1	1	Graham, Miss. Marg...	female	19.0	0	0	112053	30.0000	B42	S
888	889	0	3	Johnston, Miss. Ca...	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

```
[891 rows x 12 columns]
```

1.3 head()와 shape

```
titanic_df.head(3)
```

줄	코드	상세 설명
1	titanic_df.head(3)	head(N) 은 맨 앞 N개의 로우를 반환합니다. 인자를 생략하면 기본값 5개입니다. 반대로 뒤쪽을 볼 때는 tail(N) 을 사용합니다. 대용량 데이터에서 전체를 출력하지 않고 구조만 빠르게 확인할 때 필수적인 메서드입니다.

▶ 실행 결과

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen H...	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. L...	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S

```
print('DataFrame 크기: ', titanic_df.shape)
```

줄	코드	상세 설명
1	titanic_df.shape	NumPy와 동일하게 (행 개수, 열 개수) 튜플을 반환합니다. (891, 12) → 891개의 로우(승객)와 12개의 컬럼(속성)으로 구성됨을 의미합니다. shape[0] 은 데이터 건수, shape[1] 은 피쳐 개수로 자주 활용됩니다.

▶ 실행 결과

```
DataFrame 크기: (891, 12)
```

1.4 info() - 데이터 요약 정보

```
titanic_df.info()
```

줄	코드	상세 설명
---	----	-------

1	titanic_df.info()	DataFrame의 종합 요약 정보 를 출력합니다. ① 총 로우 수와 인덱스 범위 (RangeIndex: 891 entries), ② 컬럼별 Non-Null Count (결측이 아닌 값의 개수), ③ 컬럼별 Dtype (int64/float64/object), ④ 타입별 컬럼 수와 메모리 사용량을 보여 줍니다. 데이터 로딩 직후 가장 먼저 호출하는 필수 진단 메서드 입니다.
---	-------------------	--

▶ 실행 결과

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

■ 결과 해석 - 결측치 즉시 파악하기

전체 891건인데 **Age는 714건**(177건 결측), **Cabin은 204건**(687건 결측), **Embarked는 889건**(2건 결측)입니다. 이 세 컬럼이 전처리 대상임을 `info()` 한 번으로 알 수 있습니다. `object` 타입은 판다스에서 **문자열(또는 혼합 타입)**을 의미하며, 머신러닝 학습 전에 반드시 숫자로 인코딩해야 합니다.

1.5 describe() - 기술 통계량

```
titanic_df.describe()
```

줄	코드	상세 설명
1	titanic_df.describe()	숫자형(int, float) 컬럼만 대상으로 기술 통계량을 계산합니다. count(건수), mean(평균), std(표준편차), min(최솟값), 25%/50%/75%(사분위수), max(최댓값) 8가지를 제공합니다. 문자열(object) 컬럼은 자동 제외됩니다.

▶ 실행 결과

```
count    PassengerId  Survived  Pclass     Age     SibSp  Parch    Fare
count    891.000000   891.000000  891.000000  714.000000  891.000000  891.000000  891.000000
mean      446.000000     0.383838    2.308642   29.699118    0.523008    0.381594   32.204208
std       257.353842     0.486592    0.836071   14.526497    1.102743    0.806057   49.693429
min         1.000000     0.000000    1.000000    0.420000    0.000000    0.000000    0.000000
25%       223.500000     0.000000    2.000000   20.125000    0.000000    0.000000    7.910400
50%       446.000000     0.000000    3.000000   28.000000    0.000000    0.000000   14.454200
75%       668.500000     1.000000    3.000000   38.000000    1.000000    0.000000   31.000000
max       891.000000     1.000000    3.000000   80.000000    8.000000    6.000000  512.329200
```

■ 결과 해석 - 카테고리 컬럼 판별과 이상치 탐지

- **Survived**: min=0, max=1 → 0/1 두 값만 가지는 **이진 분류 레이블**임을 알 수 있습니다. 평균 0.3838은 **생존율 약 38.4%**를 의미합니다.
- **Pclass**: min=1, max=3이고 사분위수가 2, 3, 3 → 1~3의 **카테고리형 컬럼**으로 추정됩니다.
- **Age**: count가 714로 다른 컬럼(891)보다 작아 **결측치 존재**가 재확인됩니다. min=0.42는 생후 5개월 아기입니다.
- **Fare**: 75%가 31인데 max가 512 → **극단적인 이상치(outlier)**가 존재하며 분포가 심하게 오른쪽으로 치우쳐 있습니다.

1.6 value_counts() - 값 분포 확인

```
value_counts = titanic_df['Pclass'].value_counts()
print(value_counts)
```

줄	코드	상세 설명
1	titanic_df['Pclass'].value_counts()	① titanic_df['Pclass'] 로 Pclass 컬럼을 Series 로 추출하고, ② value_counts() 로 동일한 값이 몇 건씩 있는지 집계합니다. 결과는 건수 기준 내림차순으로 자동 정렬 된 Series입니다. 카테고리형 컬럼의 분포를 파악하는 가장 기본적인 방법입니다.
2	print(value_counts)	3등급 491명, 1등급 216명, 2등급 184명. 3등급 승객이 절반 이상임을 확인합니다.

▶ 실행 결과

```
Pclass
3    491
1    216
2    184
Name: count, dtype: int64
```

⚠ 버전 차이 주의

pandas 2.x부터 value_counts() 결과 Series의 이름이 **count** 로, 인덱스 이름이 원본 컬럼명(**Pclass**)으로 표시됩니다. 구 버전(1.x)에서는 **Name: Pclass** 로만 표시되었습니다. 값 자체는 동일합니다.

1.7 Series와 DataFrame의 차이

```
titanic_pclass = titanic_df['Pclass']
print(type(titanic_pclass))
```

줄	코드	상세 설명
1	titanic_pclass = titanic_df['Pclass']	대괄호에 문자열 컬럼명 하나 를 넣으면 해당 컬럼이 추출됩니다.
2	print(type(titanic_pclass))	결과는 <class 'pandas.core.series.Series'> . 단일 컬럼 추출 결과는 DataFrame이 아니라 Series 입니다. 참고로 titanic_df[['Pclass']] 처럼 리스트로 감싸면 DataFrame 이 반환됩니다 — 이 차이는 매우 자주 혼동되는 부분입니다.

▶ 실행 결과

```
<class 'pandas.core.series.Series'>
```

```
titanic_pclass.head()
```

줄	코드	상세 설명
1	titanic_pclass.head()	Series에도 <code>head()</code> 를 쓸 수 있습니다. 출력은 왼쪽에 인덱스, 오른쪽에 값 인 2열 구조이며, 맨 아래에 <code>Name</code> (Series 이름)과 <code>dtype</code> 이 함께 표시됩니다. 이 형태가 Series의 전형적인 출력 모습입니다.

▶ 실행 결과

```
0    3
1    1
2    3
3    1
4    3
Name: Pclass, dtype: int64
```

```
value_counts = titanic_df['Pclass'].value_counts()
print(type(value_counts))
print(value_counts)
```

줄	코드	상세 설명
2	print(type(value_counts))	<code>value_counts()</code> 의 반환 타입도 Series 입니다. 이때 인덱스는 원본의 고유 값(3, 1, 2), 값은 해당 건수(491, 216, 184)가 됩니다. 인덱스가 반드시 0,1,2... 숫자일 필요는 없다는 점을 보여 주는 좋은 예입니다.

▶ 실행 결과

```
<class 'pandas.core.series.Series'>
Pclass
3    491
1    216
2    184
Name: count, dtype: int64
```

1.8 dropna 옵션과 결측치 카운트

```
print('titanic_df 데이터 건수:', titanic_df.shape[0])
print('기본 설정인 dropna=True로 value_counts()')
# value_counts()는 디폴트로 dropna=True이므로 value_counts(dropna=True)와 동일.
print(titanic_df['Embarked'].value_counts())
print(titanic_df['Embarked'].value_counts(dropna=False))
```

줄	코드	상세 설명
1	titanic_df.shape[0]	전체 데이터 건수 891을 출력합니다. 아래 집계 결과와 비교하기 위한 기준값입니다.
4	titanic_df['Embarked'].value_counts()	기본값 <code>dropna=True</code> 가 적용되어 NaN(결측치) 은 집계에서 제외됩니다. $S(644)+C(168)+Q(77) = 889$ 로 전체 891보다 2건이 적습니다.

5	... value_counts(dropna=False)	<code>dropna=False</code> 를 주면 NaN도 하나의 값으로 취급 하여 집계합니다. NaN 2건이 함께 표시되며 합계가 891이 됩니다. 결측치 규모를 값 분포와 함께 확인할 때 유용합니다.
---	--------------------------------	---

▶ 실행 결과

```
titanic_df 데이터 건수: 891
기본 설정인 dropna=True로 value_counts()
Embarked
S    644
C    168
Q     77
Name: count, dtype: int64
Embarked
S    644
C    168
Q     77
NaN     2
Name: count, dtype: int64
```

2. DataFrame과 리스트·딕셔너리·ndarray 상호 변환

사이킷런의 많은 API는 ndarray를 입력으로 받고, 판다스는 DataFrame으로 데이터를 다룹니다. 따라서 **두 자료구조 간의 변환**은 머신러닝 코드에서 끊임없이 등장합니다.

2.1 1차원 리스트/ndarray → DataFrame

```
import numpy as np

col_name1=['col1']
list1 = [1, 2, 3]
array1 = np.array(list1)
print('array1 shape:', array1.shape )
# 리스트를 이용해 DataFrame 생성.
df_list1 = pd.DataFrame(list1, columns=col_name1)
print('1차원 리스트로 만든 DataFrame:\n', df_list1)
# 넘파이 ndarray를 이용해 DataFrame 생성.
df_array1 = pd.DataFrame(array1, columns=col_name1)
print('1차원 ndarray로 만든 DataFrame:\n', df_array1)
```

줄	코드	상세 설명
3	<code>col_name1=['col1']</code>	DataFrame에 부여할 컬럼명 리스트 를 준비합니다. 컬럼이 1개이므로 원소도 1개입니다.
4~5	<code>list1 = [1,2,3]; array1 = np.array(list1)</code>	동일한 데이터를 리스트와 ndarray 두 형태로 준비합니다.
6	<code>print('array1 shape:', array1.shape)</code>	<code>(3,)</code> → 3개 원소의 1차원 배열임을 확인합니다.
8	<code>df_list1 = pd.DataFrame(list1, columns=col_name1)</code>	<code>pd.DataFrame()</code> 생성자의 첫 인자에 데이터, <code>columns</code> 인자에 컬럼명을 전달합니다. 1차원 데이터는 1개 컬럼 으로, 각 원소가 별도의 로우가 됩니다. 인덱스는 0,1,2가 자동 부여됩니다.
11	<code>df_array1 = pd.DataFrame(array1, columns=col_name1)</code>	ndarray를 넣어도 결과가 완전히 동일 합니다. 판다스는 리스트와 ndarray를 같은 방식으로 처리합니다.

▶ 실행 결과

```
array1 shape: (3,)
1차원 리스트로 만든 DataFrame:
   col1
0     1
1     2
2     3
1차원 ndarray로 만든 DataFrame:
   col1
0     1
1     2
2     3
```

⚠ 컬럼명 개수는 반드시 일치해야 합니다

`columns`에 전달하는 리스트의 길이가 데이터의 열 수와 다르면 `ValueError: Shape of passed values ...` 오류가 발생합니다.

2.2 2차원 데이터 → DataFrame

```
# 3개의 컬럼명이 필요함.
col_name2=['col1', 'col2', 'col3']

# 2행x3열 형태의 리스트와 ndarray 생성한 뒤 이를 DataFrame으로 변환.
list2 = [[1, 2, 3],
         [11, 12, 13]]
array2 = np.array(list2)
print('array2 shape:', array2.shape)
df_list2 = pd.DataFrame(list2, columns=col_name2)
print('2차원 리스트로 만든 DataFrame:\n', df_list2)
df_array2 = pd.DataFrame(array2, columns=col_name2)
print('2차원 ndarray로 만든 DataFrame:\n', df_array2)
```

줄	코드	상세 설명
2	<code>col_name2=['col1', 'col2', 'col3']</code>	열이 3개이므로 컬럼명도 3개 를 준비합니다.
5~7	<code>list2 = [[1,2,3],[11,12,13]]</code>	중첩 리스트로 2행 3열 데이터를 만듭니다. 안쪽 리스트 하나가 하나의 로우 에 대응합니다.
8	<code>print('array2 shape:', array2.shape)</code>	<code>(2, 3)</code> → 2행 3열임을 확인합니다.
9	<code>df_list2 = pd.DataFrame(list2, columns=col_name2)</code>	2차원 리스트를 그대로 넣으면 2행 3열의 DataFrame 이 생성됩니다. 첫 행 <code>[1,2,3]</code> , 둘째 행 <code>[11,12,13]</code> .
11	<code>df_array2 = pd.DataFrame(array2, columns=col_name2)</code>	ndarray도 동일한 결과를 만듭니다.

▶ 실행 결과

```
array2 shape: (2, 3)
2차원 리스트로 만든 DataFrame:
   col1  col2  col3
0     1     2     3
1    11    12    13
2차원 ndarray로 만든 DataFrame:
   col1  col2  col3
0     1     2     3
1    11    12    13
```

2.3 딕셔너리 → DataFrame

```
# Key는 컬럼명으로 매핑, Value는 리스트 형(또는 ndarray)
dict = {'col1':[1, 11], 'col2':[2, 22], 'col3':[3, 33]}
df_dict = pd.DataFrame(dict)
print('딕셔너리로 만든 DataFrame:\n', df_dict)
```

줄	코드	상세 설명
2	dict = {'col1':[1,11], 'col2':[2,22], 'col3':[3,33]}	딕셔너리의 Key가 컬럼명, Value(리스트)가 해당 컬럼의 데이터 로 매핑됩니다. 리스트와 달리 열 단위 로 데이터를 정의한다는 점이 핵심 차이입니다.
3	df_dict = pd.DataFrame(dict)	딕셔너리를 넣으면 columns 인자를 따로 줄 필요가 없습니다 (Key가 이미 컬럼명 역할). 결과는 col1=[1,11], col2=[2,22], col3=[3,33]인 2행 3열 DataFrame입니다.

▶ 실행 결과

```
딕셔너리로 만든 DataFrame:
   col1  col2  col3
0      1     2     3
1     11    22    33
```

⚠ 코딩 스타일 주의

예제에서 변수명으로 사용한 **dict** 는 파이썬 **내장 함수명**입니다. 이렇게 쓰면 해당 스코프에서 **dict()** 함수를 쓸 수 없게 되므로, 실무에서는 **data_dict** 같은 다른 이름을 사용하는 것이 바람직합니다.

2.4 DataFrame → ndarray / 리스트 / 딕셔너리

```
# DataFrame을 ndarray로 변환
array3 = df_dict.values
print('df_dict.values 타입:', type(array3), 'df_dict.values shape:', array3.shape)
print(array3)
```

줄	코드	상세 설명
2	array3 = df_dict.values	values 는 메서드가 아니라 속성 이므로 괄호를 붙이지 않습니다. DataFrame의 데이터 부분만 ndarray로 반환 하며, 컬럼명과 인덱스는 제외 됩니다. 사이킷런에 데이터를 넘길 때 가장 많이 쓰이는 변환입니다.
3~4	print(type / shape / 값)	타입은 numpy.ndarray , shape는 (2, 3) 으로 원본 DataFrame의 크기와 동일합니다.

▶ 실행 결과

```
df_dict.values 타입: <class 'numpy.ndarray'> df_dict.values shape: (2, 3)
[[ 1  2  3]
 [11 22 33]]
```

```
# DataFrame을 리스트로 변환
list3 = df_dict.values.tolist()
print('df_dict.values.tolist() 타입:', type(list3))
print(list3)
```

```
# DataFrame을 딕셔너리로 변환
```

```
dict3 = df_dict.to_dict('list')
print('\n df_dict.to_dict() 타입:', type(dict3))
print(dict3)
```

줄	코드	상세 설명
2	list3 = df_dict.values.tolist()	DataFrame → ndarray(values) → 리스트(tolist())의 2단계 변환입니다. DataFrame에는 tolist가 없으므로 반드시 values를 거쳐야 합니다. 결과는 중첩 리스트입니다.
7	dict3 = df_dict.to_dict('list')	to_dict()의 인자 'list'는 각 컬럼의 값들을 리스트로 묶는 형식을 지정합니다. 결과는 {컬럼명: [값들]} 구조입니다. 다른 옵션으로 'records' (로 우별 딕셔너리 리스트), 'index' 등이 있습니다.

▶ 실행 결과

```
df_dict.values.tolist() 타입: <class 'list'>
[[1, 2, 3], [11, 22, 33]]

df_dict.to_dict() 타입: <class 'dict'>
{'col1': [1, 11], 'col2': [2, 22], 'col3': [3, 33]}
```

3. DataFrame의 컬럼 데이터 세트 생성과 수정

DataFrame에 새로운 컬럼을 추가하거나 기존 컬럼을 수정하는 것은 파이썬 딕셔너리에 키를 추가하듯 매우 간단합니다.

```
titanic_df['Age_0']=0
titanic_df.head(3)
```

줄	코드	상세 설명
1	titanic_df['Age_0']=0	존재하지 않는 컬럼명에 값을 할당하면 새 컬럼이 맨 뒤에 추가됩니다. 스칼라 값 0을 대입하면 모든 로우에 0이 브로드캐스팅되어 채워집니다.
2	titanic_df.head(3)	맨 오른쪽에 Age_0 컬럼이 0으로 채워져 추가된 것을 확인합니다.

▶ 실행 결과 (주요 컬럼만 표시)

```
PassengerId  Survived  Pclass   Age  SibSp  Parch  Age_0
0            1         0       3  22.0     1     0     0
1            2         1       1  38.0     1     0     0
2            3         1       3  26.0     0     0     0
```

```
titanic_df['Age_by_10'] = titanic_df['Age']*10
titanic_df['Family_No'] = titanic_df['SibSp'] + titanic_df['Parch']+1
titanic_df.head(3)
```

줄	코드	상세 설명
1	titanic_df['Age_by_10'] = titanic_df['Age']*10	기존 컬럼(Series)에 산술 연산을 적용한 결과로 새 컬럼을 만듭니다. Series 연산은 모든 원소에 일괄 적용(벡터화)되므로 for 루프가 필요 없습니다. 22.0 → 220.0.
2	titanic_df['Family_No'] = titanic_df['SibSp'] + titanic_df['Parch']+1	두 컬럼끼리 같은 인덱스 위치의 값이 서로 더해집니다. SibSp(형제·배우자 수) + Parch(부모·자녀 수) + 1(본인) = 가족 총 인원이라는 새로운 파생 피처를 생성한 것입니다. 이런 피처 엔지니어링이 모델 성능 향상의 핵심입니다.

▶ 실행 결과 (주요 컬럼만 표시)

	PassengerId	Age	SibSp	Parch	Age_0	Age_by_10	Family_No
0	1	22.0	1	0	0	220.0	2
1	2	38.0	1	0	0	380.0	2
2	3	26.0	0	0	0	260.0	1

```
titanic_df['Age_by_10'] = titanic_df['Age_by_10'] + 100
titanic_df.head(3)
```

줄	코드	상세 설명
1	titanic_df['Age_by_10'] = titanic_df['Age_by_10'] + 100	이미 존재하는 컬럼명에 할당하면 새로 추가되는 것이 아니라 기존 값이 갱신 됩니다. 모든 값에 100이 더해져 220.0 → 320.0이 됩니다.

▶ 실행 결과 (주요 컬럼만 표시)

	PassengerId	Age	Age_0	Age_by_10	Family_No
0	1	22.0	0	320.0	2
1	2	38.0	0	480.0	2
2	3	26.0	0	360.0	1

■ 핵심 정리

새 컬럼 추가와 기존 컬럼 수정은 **문법이 완전히 동일**합니다. 컬럼명 존재 여부에 따라 동작이 갈리므로, 오타로 인해 의도치 않은 컬럼이 생기지 않도록 주의해야 합니다.

4. DataFrame 데이터 삭제 - drop()

drop()의 핵심 파라미터는 **labels**(삭제 대상), **axis**(방향), **inplace**(원본 반영 여부) 세 가지입니다.

파라미터	값	의미
axis	axis=1	컬럼을 삭제합니다. 실무에서 대부분 이 경우입니다.
	axis=0	로우(행)를 삭제합니다. 이상치 레코드 제거 등에 사용합니다.
inplace	False(기본값)	원본은 유지하고 삭제된 새 DataFrame을 반환합니다.
	True	원본 자체를 변경하고 None을 반환합니다.

4.1 컬럼 삭제 (axis=1, inplace=False)

```
titanic_drop_df = titanic_df.drop('Age_0', axis=1)
titanic_drop_df.head(3)
```

줄	코드	상세 설명
1	titanic_drop_df = titanic_df.drop('Age_0', axis=1)	첫 인자에 삭제할 컬럼명 , axis=1 로 컬럼 방향 삭제를 지정합니다. inplace 를 생략했으므로 기본값 False가 적용되어 원본은 그대로 이고 삭제된 새 DataFrame이 반환됩니다.
2	titanic_drop_df.head(3)	반환된 DataFrame에는 Age_0 컬럼이 없습니다.

▶ 실행 결과 (주요 컬럼만 표시)

```

PassengerId  Age  Age_by_10  Family_No
0            1  22.0      320.0         2
1            2  38.0      480.0         2
2            3  26.0      360.0         1

```

```
titanic_df.head(3)
```

줄	코드	상세 설명
1	titanic_df.head(3)	원본을 다시 확인하는 것이 이 셀의 목적입니다. Age_0 컬럼이 여전히 남아 있음을 통해 inplace=False 가 원본을 건드리지 않음을 검증합니다.

▶ 실행 결과 (주요 컬럼만 표시)

```

PassengerId  Age  Age_0  Age_by_10  Family_No
0            1  22.0    0      320.0         2
1            2  38.0    0      480.0         2
2            3  26.0    0      360.0         1

```

4.2 inplace=True와 여러 컬럼 동시 삭제

```

drop_result = titanic_df.drop(['Age_0', 'Age_by_10', 'Family_No'], axis=1, inplace=True)
print(' inplace=True 로 drop 후 반환된 값:', drop_result)
titanic_df.head(3)

```

줄	코드	상세 설명
1	titanic_df.drop([...3개...], axis=1, inplace=True)	첫 인자에 리스트를 주면 여러 컬럼을 한 번에 삭제할 수 있습니다. inplace=True 이므로 titanic_df 자체가 변경됩니다.
2	print(' ... 반환된 값:', drop_result)	출력은 None입니다. inplace=True 일 때 반환값이 없다는 사실을 모르고 df = df.drop(..., inplace=True) 처럼 쓰면 df가 None이 되어 이후 코드가 모두 깨지는 대표적인 실수가 발생합니다.
3	titanic_df.head(3)	3개 컬럼이 모두 사라지고 원래의 12개 컬럼만 남았음을 확인합니다.

▶ 실행 결과

```

inplace=True 로 drop 후 반환된 값: None
PassengerId  Survived  Pclass      Name      Sex  Age  SibSp  Parch      Ticket     Fare  Cabin  Embarked
0            1         0       3  Braund, Mr. Ow...  male  22.0    1     0      A/5 21171   7.2500   NaN      S
1            2         1       1  Cumings, Mrs. ... female  38.0    1     0      PC 17599  71.2833   C85      C
2            3         1       3  Heikkinen, Mis... female  26.0    0     0  STON/O2. 3101282  7.9250   NaN      S

```

⚠ 절대 하지 말아야 할 코드

titanic_df = titanic_df.drop('Age_0', axis=1, inplace=True) → titanic_df에 None이 대입되어 데이터가 통째로 사라집니다. inplace=True를 쓸 때는 반환값을 대입하지 않는 것이 철칙입니다.

4.3 로우 삭제 (axis=0)

```

pd.set_option('display.width', 1000)
pd.set_option('display.max_colwidth', 15)
print('#### before axis 0 drop ####')
print(titanic_df.head(3))

```

```
titanic_df.drop([0,1,2], axis=0, inplace=True)

print('#### after axis 0 drop ####')
print(titanic_df.head(3))
```

줄	코드	상세 설명
1	pd.set_option('display.width', 1000)	판다스의 출력 옵션 을 조정합니다. 한 줄에 표시할 문자 폭을 1000으로 늘려 컬럼이 줄바꿈되지 않게 합니다.
2	pd.set_option('display.max_colwidth', 15)	각 셀에 표시할 최대 문자 수 를 15로 제한합니다. Name처럼 긴 문자열이 ... 으로 줄여져 표가 깔끔해집니다. 출력 형태만 바꿀 뿐 데이터 자체는 변하지 않습니다 .
6	titanic_df.drop([0,1,2], axis=0, inplace=True)	axis=0 이므로 리스트 [0,1,2]는 컬럼명이 아니라 인덱스 값 으로 해석되어 해당 3개 로우가 삭제 됩니다. inplace=True로 원본에 즉시 반영됩니다.
9	print(titanic_df.head(3))	삭제 후 head(3)의 결과가 인덱스 3, 4, 5 부터 시작합니다. 여기서 중요한 점은 인덱스가 0부터 재부여되지 않고 원래 값을 유지 한다는 것입니다.

▶ 실행 결과

```
#### before axis 0 drop ####
  PassengerId  Survived  Pclass     Name    Sex  Age  SibSp  Parch    Ticket   Fare Cabin Embarked
0         1         0         3  Braund, Mr....  male  22.0    1     0      A/5 21171   7.2500   NaN      S
1         2         1         1  Cumings, Mr... female  38.0    1     0      PC 17599  71.2833   C85      C
2         3         1         3  Heikkinen, ... female  26.0    0     0  STON/O2. 31...   7.9250   NaN      S
#### after axis 0 drop ####
  PassengerId  Survived  Pclass     Name    Sex  Age  SibSp  Parch    Ticket   Fare Cabin Embarked
3         4         1         1  Futrelle, M... female  35.0    1     0  113803  53.1000  C123      S
4         5         0         3  Allen, Mr. ...  male  35.0    0     0  373450   8.0500   NaN      S
5         6         0         3  Moran, Mr. ...  male   NaN    0     0  330877   8.4583   NaN      Q
```

■ axis 개념 정리

판다스에서 **axis=0**은 **로우 방향(위→아래)**, **axis=1**은 **컬럼 방향(좌→우)**을 의미합니다. **drop()**에서는 "그 축에 해당하는 것을 지운다"로, **sum()** 같은 집계에서는 "그 축을 따라 계산한다"로 해석하면 됩니다. axis 기본값은 0이므로 **컬럼 삭제 시에는 axis=1을 반드시 명시**해야 합니다.

5. Index 객체

5.1 Index 객체의 추출과 특성

```
# 원본 파일 재 로딩
titanic_df = pd.read_csv('titanic_train.csv')
# Index 객체 추출
indexes = titanic_df.index
print(indexes)
# Index 객체를 실제 값 array로 변환
print('Index 객체 array값:\n', indexes.values)
```

줄	코드	상세 설명
---	----	-------

2	titanic_df = pd.read_csv('titanic_train.csv')	앞에서 로우/컬럼을 삭제했으므로 원본을 다시 로딩 해 초기 상태로 되돌립니다.
4	indexes = titanic_df.index	<code>index</code> 속성으로 Index 객체 를 추출합니다. 출력은 <code>RangeIndex(start=0, stop=891, step=1)</code> 로, 모든 값을 저장하지 않고 시작·끝·간격만 기억하는 메모리 효율적인 형태 입니다.
6	print(indexes.values)	Index 객체의 <code>values</code> 속성은 실제 인덱스 값들을 1차원 ndarray 로 반환합니다. 0부터 890까지 891개 값이 출력됩니다.

▶ 실행 결과 (일부 생략)

```
RangeIndex(start=0, stop=891, step=1)
Index 객체 array값:
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17
 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35
 ... (중략) ...
882 883 884 885 886 887 888 889 890]
```

```
print(type(indexes.values))
print(indexes.values.shape)
print(indexes[:5].values)
print(indexes.values[:5])
print(indexes[6])
```

줄	코드	상세 설명
1	print(type(indexes.values))	<code><class 'numpy.ndarray'></code> → Index의 값은 내부적으로 ndarray로 관리됩니다.
2	print(indexes.values.shape)	<code>(891,)</code> → 891개 원소의 1차원 배열입니다.
3	print(indexes[:5].values)	Index 객체를 먼저 슬라이싱한 뒤 values로 변환 합니다. → <code>[0 1 2 3 4]</code>
4	print(indexes.values[:5])	ndarray로 먼저 변환한 뒤 슬라이싱 합니다. 결과는 3번 줄과 동일 → Index 객체도 ndarray처럼 슬라이싱이 가능함을 보여 줍니다.
5	print(indexes[6])	단일 위치 접근도 가능하며 값 6이 출력됩니다.

▶ 실행 결과

```
<class 'numpy.ndarray'>
(891,)
[0 1 2 3 4]
[0 1 2 3 4]
6
```

```
indexes[0] = 5
```

줄	코드	상세 설명
1	indexes[0] = 5	오류가 발생합니다. Index 객체는 한 번 만들어진다면 변경할 수 없는 (immutable) 객체이기 때문입니다. 이는 인덱스가 레코드를 식별하는 키 역할을 하므로 임의 변경으로 데이터 정합성이 깨지는 것을 막기 위한 판다스의 설계 원칙입니다. 인덱스를 바꾸려면 <code>reset_index()</code> 나 <code>set_index()</code> 를 사용해야 합니다.

▶ 실행 결과 (오류)

TypeError: Index does not support mutable operations

5.2 Series 연산에서 Index의 역할

```
series_fair = titanic_df['Fare']
print('Fair Series max 값:', series_fair.max())
print('Fair Series sum 값:', series_fair.sum())
print('sum() Fair Series:', sum(series_fair))
print('Fair Series + 3:\n', (series_fair + 3).head(3) )
```

줄	코드	상세 설명
1	series_fair = titanic_df['Fare']	요금(Fare) 컬럼을 Series로 추출합니다.
2	series_fair.max()	판다스 내장 <code>max()</code> 메서드로 최댓값 512.3292를 구합니다.
3	series_fair.sum()	합계 28693.9493. 판다스 메서드는 NaN을 자동으로 제외 하고 계산합니다.
4	sum(series_fair)	파이썬 내장 <code>sum()</code> 함수로도 계산됩니다. 결과가 미세하게 다른 (28693.949299999967) 이유는 부동소수점 누적 연산 순서 차이 때문입니다. 대용량에서는 판다스 메서드가 훨씬 빠르므로 <code>series.sum()</code> 을 사용하는 것이 좋습니다.
5	(series_fair + 3).head(3)	Series에 스칼라를 더하면 모든 원소에 일괄 적용 됩니다(7.25 → 10.25). 핵심은 결과가 단순 배열이 아니라 인덱스가 그대로 유지된 Series 라는 점입니다. 판다스의 모든 연산은 인덱스를 기준으로 정렬(alignment) 되어 수행됩니다.

▶ 실행 결과

```
Fair Series max 값: 512.3292
Fair Series sum 값: 28693.9493
sum() Fair Series: 28693.949299999967
Fair Series + 3:
0    10.2500
1    74.2833
2    10.9250
Name: Fare, dtype: float64
```

5.3 reset_index()

```
titanic_reset_df = titanic_df.reset_index(inplace=False)
titanic_reset_df.head(3)
```

줄	코드	상세 설명
1	titanic_df.reset_index(inplace=False)	<code>reset_index()</code> 는 ① 기존 인덱스를 'index'라는 새 컬럼으로 추가 하고, ② 인덱스를 0부터 시작하는 연속 숫자로 새로 부여 합니다. 로우 삭제나 정렬 후 인덱스가 뒤죽박죽일 때 정리하는 용도로 쓰입니다. 기존 인덱스가 필요 없다면 <code>drop=True</code> 옵션으로 컬럼 추가 없이 재설정만 할 수 있습니다.

▶ 실행 결과

	index	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	0	1	0	3	Braund, Mr. Ow...	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	1	2	1	1	Cumings, Mrs. ...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	2	3	1	3	Heikkinen, Mis...	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S

```
print('### before reset_index ###')
value_counts = titanic_df['Pclass'].value_counts()
print(value_counts)
print('value_counts 객체 변수 타입:', type(value_counts))

new_value_counts = value_counts.reset_index(inplace=False)
print('### After reset_index ###')
print(new_value_counts)
print('new_value_counts 객체 변수 타입:', type(new_value_counts))
```

줄	코드	상세 설명
2~4	value_counts = ...; print(type(...))	value_counts 결과는 Series 이며, 인덱스가 3, 1, 2(Pclass 값)라는 비연속·비정렬 형태입니다.
6	new_value_counts = value_counts.reset_index(inplace=False)	Series에 reset_index를 적용하면 인덱스가 컬럼으로 바뀌면서 타입이 Series → DataFrame으로 변환 됩니다. pandas 2.x 기준 컬럼명은 Pclass 와 count 가 됩니다.
9	print(type(new_value_counts))	DataFrame 으로 바뀐 것을 확인합니다. 집계 결과를 다른 데이터와 병합(merge)하거나 시각화 입력으로 쓸 때 이 변환이 필요합니다.

▶ 실행 결과

```
### before reset_index ###
Pclass
3    491
1    216
2    184
Name: count, dtype: int64
value_counts 객체 변수 타입: <class 'pandas.core.series.Series'>
### After reset_index ###
   Pclass  count
0       3    491
1       1    216
2       2    184
new_value_counts 객체 변수 타입: <class 'pandas.core.frame.DataFrame'>
```

⚠ 버전 차이

pandas 1.x에서는 이 결과의 컬럼명이 **index**와 **Pclass**였습니다. 2.x부터 **Pclass**와 **count**로 더 직관적으로 변경되었으므로, 오래된 교재 코드를 실행할 때는 컬럼명 참조 부분을 확인해야 합니다.

6. 데이터 셀렉션 및 필터링

판다스는 데이터 선택 방식이 여러 가지여서 초보자가 가장 혼란스러워하는 부분입니다. 아래 표로 전체 지도를 먼저 파악하고 각각을 학습합니다.

방식	기준	사용 예	결과
df['컬럼명']	컬럼명	df['Pclass']	Series
df[['컬럼1', '컬럼2']]	컬럼명 리스트	df[['Survived', 'Pclass']]	DataFrame
df[0:2]	로우 슬라이싱	인덱스 0~1행	DataFrame

<code>df[불린 Series]</code>	조건	<code>df[df['Age']>60]</code>	DataFrame
<code>df.iloc[행, 열]</code>	위치(정수)	<code>df.iloc[0, 0]</code>	값/Series/DataFrame
<code>df.loc[행, 열]</code>	명칭(레이블)	<code>df.loc['one', 'Name']</code>	값/Series/DataFrame

6.1 [] 연산자

```
print('단일 컬럼 데이터 추출:\n', titanic_df[ 'Pclass' ].head(3))
print('\n여러 컬럼들의 데이터 추출:\n', titanic_df[ ['Survived', 'Pclass'] ].head(3))
print('[ ] 안에 숫자 index는 KeyError 오류 발생:\n', titanic_df[0])
```

줄	코드	상세 설명
1	<code>titanic_df['Pclass'].head(3)</code>	컬럼명 문자열 1개 → Series 반환.
2	<code>titanic_df[['Survived', 'Pclass']].head(3)</code>	컬럼명 리스트 → 여러 컬럼을 가진 DataFrame 반환. 대괄호가 두 겹인 이유는 "리스트를 대괄호 안에 넣었기" 때문입니다.
3	<code>titanic_df[0]</code>	KeyError가 발생합니다. DataFrame의 [] 안에 들어가는 숫자는 위치가 아니라 컬럼명으로 해석되는데, '0'이라는 컬럼이 없기 때문입니다. 위치로 접근하려면 반드시 <code>iloc[]</code> 를 사용해야 합니다.

▶ 실행 결과

```
단일 컬럼 데이터 추출:
0    3
1    1
2    3
Name: Pclass, dtype: int64
```

```
여러 컬럼들의 데이터 추출:
   Survived  Pclass
0         0        3
1         1        1
2         1        3
```

```
KeyError: 0
```

```
titanic_df[0:2]
```

줄	코드	상세 설명
1	<code>titanic_df[0:2]</code>	같은 []인데 슬라이싱을 넣으면 로우 기준으로 동작하여 0~1번 행이 반환됩니다. <code>df[0]</code> 은 오류인데 <code>df[0:2]</code> 는 동작하는 이 비일관성 때문에, 실무에서는 [] 슬라이싱보다 <code>iloc / loc</code> 사용을 권장합니다.

▶ 실행 결과

```
PassengerId  Survived  Pclass      Name    Sex  Age  SibSp  Parch    Ticket   Fare Cabin Embarked
0           1         0        3  Braund, Mr. Ow...  male  22.0    1     0  A/5 21171   7.2500   NaN        S
1           2         1        1  Cumings, Mrs. ... female  38.0    1     0    PC 17599  71.2833   C85        C
```

```
titanic_df[ titanic_df['Pclass'] == 3].head(3)
```

줄	코드	상세 설명
---	----	-------

1	titanic_df[titanic_df['Pclass']==3]	[] 안에 불린 Series 를 넣으면 조건이 True인 로우만 반환됩니다. 내부적으로 <code>titanic_df['Pclass']==3</code> 이 891개의 True/False Series를 만들고, 이를 필터로 사용합니다. 결과 인덱스가 0, 2, 4로 건너뛰는 것이 필터링이 적용된 증거입니다.
---	-------------------------------------	--

▶ 실행 결과

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Ow...	male	22.0	1	0	A/5 21171	7.250	NaN	S
2	3	1	3	Heikkinen, Mis...	female	26.0	0	0	STON/O2. 3101282	7.925	NaN	S
4	5	0	3	Allen, Mr. Wil...	male	35.0	0	0	373450	8.050	NaN	S

6.2 iloc[] - 위치 기반 인덱싱

```
data = {'Name': ['Chulmin', 'Eunkyoung', 'Jinwoong', 'Soobeom'],
        'Year': [2011, 2016, 2015, 2015],
        'Gender': ['Male', 'Female', 'Male', 'Male']}
data_df = pd.DataFrame(data, index=['one', 'two', 'three', 'four'])
data_df
```

줄	코드	상세 설명
1~4	data = {...}	3개 컬럼(Name, Year, Gender), 4개 로우의 딕셔너리를 만듭니다.
5	data_df = pd.DataFrame(data, index=['one', 'two', 'three', 'four'])	<code>index</code> 인자로 문자열 인덱스 를 직접 지정합니다. 이렇게 하면 위치(0,1,2,3)와 명칭('one','two',...) 이 서로 다르므로 <code>iloc</code> 과 <code>loc</code> 의 차이를 명확히 실습할 수 있습니다.

▶ 실행 결과

	Name	Year	Gender
one	Chulmin	2011	Male
two	Eunkyoung	2016	Female
three	Jinwoong	2015	Male
four	Soobeom	2015	Male

```
data_df.iloc[0, 0]
```

줄	코드	상세 설명
1	data_df.iloc[0, 0]	<code>iloc</code> 은 integer location 의 약자로 오직 정수 위치만 받습니다. [0행, 0열] → 'Chulmin'. NumPy의 2차원 인덱싱과 동일한 감각으로 사용하면 됩니다.

▶ 실행 결과

'Chulmin'

```
# 아래 코드는 오류를 발생시킵니다.
data_df.iloc[0, 'Name']

# 아래 코드는 오류를 발생시킵니다.
data_df.iloc['one', 0]
```

줄	코드	상세 설명
---	----	-------

2	<code>data_df.iloc[0, 'Name']</code>	열에 컬럼명 문자열 을 주어 오류가 발생합니다. <code>iloc</code> 은 명칭을 절대 받지 않습니다.
5	<code>data_df.iloc['one', 0]</code>	행에 문자열 인덱스 를 주어 동일한 오류가 발생합니다. <code>iloc</code> 의 규칙은 " 정수, 정수 슬라이스, 정수 리스트, 불린 배열만 허용 "입니다.

▶ 실행 결과 (오류)

```
ValueError: Location based indexing can only have [integer, integer slice (START point is INCLUDED, END point is EXCLUDED), listlike of integers, boolean array] types
```

```
print("\n 맨 마지막 칼럼 데이터[:, -1] \n", data_df.iloc[:, -1])
print("\n 맨 마지막 칼럼을 제외한 모든 데이터[:, :-1] \n", data_df.iloc[:, :-1])
```

줄	코드	상세 설명
1	<code>data_df.iloc[:, -1]</code>	행은 전체(<code>:</code>), 열은 -1(맨 마지막) → Gender 컬럼이 Series로 반환됩니다. 머신러닝에서 레이블(정답) 컬럼이 맨 뒤인 경우가 많아 자주 쓰는 패턴입니다.
2	<code>data_df.iloc[:, :-1]</code>	열을 처음부터 마지막 직전까지 선택 → 마지막 열을 제외한 피쳐 데이터 가 DataFrame으로 반환됩니다. <code>X = df.iloc[:, :-1]</code> , <code>y = df.iloc[:, -1]</code> 은 데이터/레이블 분리의 관용구입니다.

▶ 실행 결과

```
맨 마지막 칼럼 데이터[:, -1]
one      Male
two      Female
three     Male
four      Male
Name: Gender, dtype: object

맨 마지막 칼럼을 제외한 모든 데이터[:, :-1]
      Name  Year
one  Chulmin  2011
two  Eunkyung 2016
three Jinwoong 2015
four  Soobeom  2015
```

6.3 loc[] - 명칭 기반 인덱싱

```
data_df.loc['one', 'Name']
```

줄	코드	상세 설명
1	<code>data_df.loc['one', 'Name']</code>	<code>loc</code> 은 label location 으로 인덱스 값과 컬럼명 을 받습니다. 'one'행의 'Name'열 → 'Chulmin'. <code>iloc[0,0]</code> 과 결과는 같지만 지정 방식이 완전히 다릅니다 .

▶ 실행 결과

```
'Chulmin'
```

```
# 아래 코드는 오류를 발생합니다.
data_df_reset.loc[0, 'Name']
```

줄	코드	상세 설명
2	<code>data_df_reset.loc[0, 'Name']</code>	<code>data_df_reset</code> 은 인덱스를 1부터 시작 하도록 만든 DataFrame입니다. <code>loc</code> 은 위치가 아닌 명칭 으로 찾으므로 0이라는 인덱스 값이 존재하지 않아 <code>KeyError</code> 가 발생합니다. <code>data_df_reset.loc[1, 'Name']</code> 은 정상 동작하여 'Chulmin'을 반환합니다. loc의 숫자는 위치가 아니라 이름 이라는 점이 핵심입니다.

▶ 실행 결과 (오류)

```
KeyError: 0
```

```
print('위치기반 iloc slicing\n', data_df.iloc[0:1, 0], '\n')
print('명칭기반 loc slicing\n', data_df.loc['one':'two', 'Name'])
```

줄	코드	상세 설명
1	<code>data_df.iloc[0:1, 0]</code>	<code>iloc</code> 슬라이싱은 파이썬 규칙대로 끝 위치를 포함하지 않습니다 . <code>0:1</code> 은 0번 행 하나만 선택 → 'Chulmin' 1건.
2	<code>data_df.loc['one':'two', 'Name']</code>	<code>loc</code> 슬라이싱은 끝 명칭까지 포함 합니다. 'one'~'two' → 2건 이 반환됩니다. 이 포함/불포함 차이 는 판다스에서 가장 실수하기 쉬운 부분이므로 반드시 기억해야 합니다.

▶ 실행 결과

```
위치기반 iloc slicing
one    Chulmin
Name: Name, dtype: object

명칭기반 loc slicing
one    Chulmin
two    Eunkyoung
Name: Name, dtype: object
```

■ `iloc` vs `loc` 최종 정리

- **iloc** : 정수 위치 기반. 슬라이싱 시 **끝 미포함**. → `iloc[0:2]` 는 2건
- **loc** : 명칭(레이블) 기반. 슬라이싱 시 **끝 포함**. → `loc['one':'two']` 는 2건, `loc[0:2]` (숫자 인덱스일 때)는 **3건**
- 불린 인덱싱은 **loc에서만** 조건과 컬럼을 함께 지정할 수 있습니다.

6.4 불린 인덱싱

```
titanic_df = pd.read_csv('titanic_train.csv')
titanic_boolean = titanic_df[titanic_df['Age'] > 60]
print(type(titanic_boolean))
titanic_boolean
```

줄	코드	상세 설명
2	<code>titanic_df[titanic_df['Age'] > 60]</code>	나이가 60을 초과하는 승객만 필터링합니다. 891건 중 22건 이 반환됩니다. NaN인 Age는 비교 결과가 False가 되어 자동 제외됩니다.
3	<code>print(type(titanic_boolean))</code>	결과 타입은 DataFrame 으로, 원본의 모든 컬럼을 그대로 가집니다.

▶ 실행 결과 (상위 3건, 총 22건)

<class 'pandas.core.frame.DataFrame'>

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
33	34	0	2	Wheaton, Mr. Edw...	male	66.0	0	0	C.A. 24579	10.5000	NaN	S
54	55	0	1	Ostby, Mr. Engel...	male	65.0	0	1	113509	61.9792	B30	C
96	97	0	1	Goldschmidt, Mr....	male	71.0	0	0	PC 17754	34.6542	A5	C

```
titanic_df[titanic_df['Age'] > 60][['Name', 'Age']].head(3)
titanic_df.loc[titanic_df['Age'] > 60, ['Name', 'Age']].head(3)
```

줄	코드	상세 설명
1	titanic_df[조건][['Name', 'Age']]	대괄호 체이닝 방식: 먼저 조건으로 로우를 거르고, 그 결과에서 다시 컬럼을 선택합니다. 동작은 하지만 중간 복사본이 생겨 값을 수정할 때 SettingWithCopyWarning 이 발생할 수 있습니다.
2	titanic_df.loc[조건, ['Name', 'Age']]	loc 방식(권장): 로우 조건과 컬럼 선택을 한 번에 지정합니다. 결과는 위와 동일하지만 더 명확하고 안전하며 성능도 좋습니다. 실무에서는 이 방식을 사용하세요.

▶ 실행 결과 (두 코드 모두 동일)

	Name	Age
33	Wheaton, Mr. Edward H	66.0
54	Ostby, Mr. Engelhart Cornelius	65.0
96	Goldschmidt, Mr. George B	71.0

```
titanic_df[ (titanic_df['Age'] > 60) & (titanic_df['Pclass']==1) & (titanic_df['Sex']=='female')]
```

줄	코드	상세 설명
1	df[(조건1) & (조건2) & (조건3)]	복합 조건을 결합합니다. 두 가지 규칙을 반드시 지켜야 합니다: ① 각 조건을 괄호로 감쌀 것 — 파이썬에서 & 가 비교 연산자보다 우선순위가 높아 괄호가 없으면 오류가 납니다. ② and/or/not이 아닌 &, , ~를 쓸 것 — 파이썬 키워드는 Series 전체의 진리값을 요구해 오류가 발생합니다. 결과: 60세 초과 + 1등급 + 여성 = 2명 .

▶ 실행 결과

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
275	276	1	1	Andrews, Miss. K...	female	63.0	1	0	13502	77.9583	D7	S
829	830	1	1	Stone, Mrs. Geor...	female	62.0	0	0	113572	80.0000	B28	NaN

```
cond1 = titanic_df['Age'] > 60
cond2 = titanic_df['Pclass']==1
cond3 = titanic_df['Sex']=='female'
titanic_df[ cond1 & cond2 & cond3]
```

줄	코드	상세 설명
1~3	cond1 / cond2 / cond3	각 조건(불린 Series)을 변수에 미리 담아 둡니다 . 조건이 3개 이상이거나 조건식이 길어질 때 코드 가독성이 크게 향상됩니다.
4	titanic_df[cond1 & cond2 & cond3]	변수로 결합하면 괄호가 불필요 해집니다. 결과는 앞의 한 줄 코드와 완전히 동일 합니다. 복잡한 필터링에는 이 방식을 권장합니다.

▶ 실행 결과

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
275	276	1	1	Andrews, Miss. K...	female	63.0	1	0	13502	77.9583	D7	S
829	830	1	1	Stone, Mrs. Geor...	female	62.0	0	0	113572	80.0000	B28	NaN

7. 정렬, Aggregation 함수, GroupBy

7.1 sort_values() - DataFrame·Series 정렬

```
titanic_sorted = titanic_df.sort_values(by=['Name'])
titanic_sorted.head(3)
```

줄	코드	상세 설명
1	titanic_df.sort_values(by=['Name'])	SQL의 ORDER BY와 동일한 역할입니다. by 에 정렬 기준 컬럼(리스트)을 지정합니다. 기본값 ascending=True 가 적용되어 오름차순(A→Z) 으로 정렬됩니다. 원본은 변경되지 않고 정렬된 새 DataFrame이 반환되며, 각 로우의 인덱스 는 원래 값을 그대로 유지한 채 순서만 바뀝니다.

▶ 실행 결과

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
845	846	0	3	Abbing, Mr. Anthony	male	42.0	0	0	C.A. 5547	7.55	NaN	S
746	747	0	3	Abbott, Mr. Ross...	male	16.0	1	1	C.A. 2673	20.25	NaN	S
279	280	1	3	Abbott, Mrs. Sta...	female	35.0	1	1	C.A. 2673	20.25	NaN	S

```
titanic_sorted = titanic_df.sort_values(by=['Pclass', 'Name'], ascending=False)
titanic_sorted.head(3)
```

줄	코드	상세 설명
1	sort_values(by=['Pclass', 'Name'], ascending=False)	다중 컬럼 정렬 입니다. Pclass 를 먼저 정렬하고, Pclass가 같은 행들 내부에서 Name 으로 2차 정렬합니다. ascending=False 는 두 컬럼 모두 내림차순 으로 적용됩니다. 결과 첫 행이 Pclass=3이고 이름이 소문자 'v'로 시작하는 것은, 문자열 정렬이 ASCII 코드 순서 를 따라 소문자(97~)가 대문자(65~)보다 크기 때문입니다.

▶ 실행 결과

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
868	869	0	3	van Melkebeke, M...	male	NaN	0	0	345777	9.5	NaN	S
153	154	0	3	van Billiard, Mr...	male	40.5	0	2	A/5. 851	14.5	NaN	S
282	283	0	3	de Pelsmaecker, M...	male	16.0	0	0	345778	9.5	NaN	S

■ 컬럼별로 정렬 방향을 다르게 하려면

ascending 에 **리스트**를 전달합니다. 예: df.sort_values(by=['Pclass', 'Name'], ascending=[True, False]) → Pclass는 오름차순, Name은 내림차순.

7.2 Aggregation 함수

```
titanic_df.count()
```


줄	코드	상세 설명
1	<code>titanic_df.count()</code>	DataFrame에 집계 함수를 직접 호출하면 모든 컬럼에 각각 적용되어 Series로 반환됩니다. 중요한 특성은 <code>count()</code> 가 NaN을 제외한 건수만 센다는 점입니다. 그래서 Age는 714, Cabin은 204, Embarked는 889로 나타나 결측치 규모를 즉시 파악할 수 있습니다.

▶ 실행 결과

```

PassengerId    891
Survived        891
Pclass          891
Name           891
Sex            891
Age            714
SibSp          891
Parch          891
Ticket         891
Fare           891
Cabin          204
Embarked       889
dtype: int64

```

```
titanic_df[['Age', 'Fare']].mean()
```

줄	코드	상세 설명
1	<code>titanic_df[['Age', 'Fare']].mean()</code>	먼저 대상 컬럼만 선택한 뒤 <code>mean()</code> 을 호출하는 것이 일반적인 패턴입니다. 문자열 컬럼이 섞여 있으면 평균 계산이 불가능하기 때문입니다. Age 평균은 29.699118 (NaN 제외한 714건 기준), Fare 평균은 32.204208 입니다. <code>sum</code> , <code>max</code> , <code>min</code> , <code>std</code> , <code>var</code> , <code>median</code> 등도 같은 방식으로 사용합니다.

▶ 실행 결과

```

Age      29.699118
Fare     32.204208
dtype: float64

```

7.3 groupby() 활용

```

titanic_groupby = titanic_df.groupby(by='Pclass')
print(type(titanic_groupby))

```

줄	코드	상세 설명
1	<code>titanic_df.groupby(by='Pclass')</code>	SQL의 GROUP BY에 해당하며, Pclass 값(1,2,3)별로 데이터를 그룹화합니다.
2	<code>print(type(titanic_groupby))</code>	반환 타입은 DataFrame이 아니라 <code>DataFrameGroupBy</code> 입니다. 이 객체는 "그룹으로 나눌 준비만 된 중간 상태"이며, 뒤에 집계 함수를 붙여야 비로소 실제 결과가 나옵니다.

▶ 실행 결과

```
<class 'pandas.core.groupby.generic.DataFrameGroupBy'>
```

```
titanic_groupby = titanic_df.groupby('Pclass').count()
titanic_groupby
```

줄	코드	상세 설명
1	<code>titanic_df.groupby('Pclass').count()</code>	GroupBy 객체에 <code>count()</code> 를 적용하면 Pclass가 인덱스 가 되고, 나머지 모든 컬럼에 대해 그룹별 건수가 계산됩니다. SQL과의 차이는 SQL에서는 집계 대상 컬럼을 SELECT에 명시하지만, 판다스는 기본적으로 모든 컬럼에 일괄 적용 한다는 점입니다.

▶ 실행 결과

```

      PassengerId  Survived  Name  Sex  Age  SibSp  Parch  Ticket  Fare  Cabin  Embarked
Pclass
1              216         216   216   216  186    216    216    216    216    176     214
2              184         184   184   184  173    184    184    184    184     16    184
3              491         491   491   491  355    491    491    491    491     12    491
```

■ 결과 해석

Cabin 컬럼을 보면 1등급 176건, 2등급 16건, 3등급 12건으로 **객실 번호 정보가 상위 등급에 집중**되어 있습니다. 즉 Cabin의 결측은 무작위가 아니라 **등급과 관련된 구조적 결측**임을 알 수 있으며, 이런 통찰이 피처 엔지니어링의 출발점이 됩니다.

```
titanic_groupby = titanic_df.groupby('Pclass')[['PassengerId', 'Survived']].count()
titanic_groupby
```

줄	코드	상세 설명
1	<code>groupby('Pclass')[['PassengerId', 'Survived']].count()</code>	groupby 뒤에 대괄호로 컬럼을 선택 하면 해당 컬럼에만 집계 적용됩니다. 필요한 컬럼만 뽑아 결과를 간결하게 만드는 표준 패턴입니다.

▶ 실행 결과

```

      PassengerId  Survived
Pclass
1              216         216
2              184         184
3              491         491
```

```
titanic_df.groupby('Pclass')['Age'].agg([max, min])
```

줄	코드	상세 설명
1	<code>groupby('Pclass')['Age'].agg([max, min])</code>	하나의 컬럼에 여러 집계 함수를 동시에 적용하려면 <code>agg()</code> 에 리스트를 전달합니다. 결과는 max, min이 각각 컬럼이 된 DataFrame입니다. 1등급 최고령 80세, 3등급 최연소 0.42세(생후 5개월) 등을 한눈에 볼 수 있습니다.

▶ 실행 결과

```

      max  min
Pclass
1    80.0  0.92
2    70.0  0.67
3    74.0  0.42
```

⚠ 안정화 버전 권장 코드

pandas 2.x에서 위 코드처럼 파이썬 내장 함수 `max`, `min` 을 그대로 넘기면 다음 FutureWarning이 발생합니다.

```
FutureWarning: The provided callable <built-in function max> is currently using
SeriesGroupBy.max. In a future version of pandas, the provided callable will be
used directly. To keep current behavior pass the string "max" instead.
```

향후 버전 호환을 위해 문자열로 전달하는 것을 권장합니다:

```
titanic_df.groupby('Pclass')['Age'].agg(['max', 'min'])
```

```
agg_format={'Age':'max', 'SibSp':'sum', 'Fare':'mean'}
titanic_df.groupby('Pclass').agg(agg_format)
```

줄	코드	상세 설명
1	<code>agg_format={'Age':'max', 'SibSp':'sum', 'Fare':'mean'}</code>	컬럼마다 서로 다른 집계 함수를 적용하기 위해 딕셔너리를 만듭니다. Key 는 컬럼명, Value 는 집계 함수명 문자열입니다.
2	<code>titanic_df.groupby('Pclass').agg(agg_format)</code>	딕셔너리를 <code>agg()</code> 에 전달하면 Age는 최댓값, SibSp는 합계, Fare는 평균이 각각 계산됩니다. 결과 해석: 1등급 평균 요금이 84.15 로 3등급 13.68 의 약 6배이며, 3등급의 SibSp 합계(302)가 커서 대가족 단위 탑승이 많았음 을 짐작할 수 있습니다.

▶ 실행 결과

	Age	SibSp	Fare
Pclass			
1	80.0	90	84.154687
2	70.0	74	20.662183
3	74.0	302	13.675550

8. 결손 데이터 처리 - `isna( )`와 `fillna( )`

판다스는 결손 데이터를 **NaN(Not a Number)**으로 표현합니다. 대부분의 머신러닝 알고리즘은 NaN을 처리하지 못하므로, **학습 전에 반드시 다른 값으로 대체**해야 합니다.

8.1 `isna( )`로 결손 데이터 확인

```
titanic_df.isna().head(3)
```

줄	코드	상세 설명
1	<code>titanic_df.isna()</code>	모든 원소에 대해 NaN이면 True, 아니면 False 인 동일 크기의 불린 DataFrame 을 반환합니다. 결과에서 0번~2번 로우의 Cabin이 True인 것을 볼 수 있습니다. <code>isnull()</code> 은 <code>isna()</code> 의 별칭으로 완전히 동일하게 동작합니다.

▶ 실행 결과

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0		False	False	False	False	False	False	False	False	False	True	False

```
1 False False False False False False False False False False False False
2 False False False False False False False False False False True False
```

```
titanic_df.isna().sum()
```

줄	코드	상세 설명
1	titanic_df.isna().sum()	결측치 개수를 세는 표준 관용구 입니다. 원리는 True=1, False=0 으로 취급되므로, sum() 이 True의 개수 = NaN의 개수 를 계산해 줍니다. 결과: Age 177건, Cabin 687건, Embarked 2건. 전체 891건 대비 Cabin은 77%가 결측 이라 컬럼 자체를 버리거나 "정보 있음/없음" 이진 피처로 변환하는 것을 고려합니다.

▶ 실행 결과

```
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age          177
SibSp          0
Parch         0
Ticket         0
Fare           0
Cabin         687
Embarked       2
dtype: int64
```

8.2 fillna()로 Missing 데이터 대체

```
titanic_df['Cabin'] = titanic_df['Cabin'].fillna('C000')
titanic_df.head(3)
```

줄	코드	상세 설명
1	titanic_df['Cabin'] = titanic_df['Cabin'].fillna('C000')	fillna() 의 인자로 준 값으로 NaN을 대체 합니다. 여기서 핵심은 결과를 다시 대입하는 것 입니다. fillna는 기본적으로 새 Series를 반환할 뿐 원본을 바꾸지 않으므로 , 대입하지 않으면 원본은 그대로 NaN으로 남습니다. 0번·2번 로우의 Cabin이 NaN → C000으로 바뀐 것을 확인할 수 있습니다.

▶ 실행 결과

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen...	male	22.0	1	0	A/5 21171	7.2500	C000	S
1	2	1	1	Cumings, Mrs. Jo...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss....	female	26.0	0	0	STON/O2. 3101282	7.9250	C000	S

⚠ pandas 2.x 주의 - inplace 체이닝은 사용 금지

구버전 교재의 `titanic_df['Cabin'].fillna('C000', inplace=True)` 방식은 pandas 2.x에서 **ChainedAssignmentError** 경고가 발생하며 향후 동작하지 않습니다. 본문처럼 **결과를 다시 대입하는 방식**이 안정화 버전의 표준입니다.

```
titanic_df['Age'] = titanic_df['Age'].fillna(titanic_df['Age'].mean())
titanic_df['Embarked'] = titanic_df['Embarked'].fillna('S')
titanic_df.isna().sum()
```

줄	코드	상세 설명
1	titanic_df['Age'].fillna(titanic_df['Age'].mean())	숫자형 컬럼의 대표적 대체 전략인 평균값 대체(mean imputation) 입니다. 177건의 NaN이 모두 29.699118 로 채워집니다. 평균 대신 이상치에 강한 중앙값(median()) 을 쓰는 경우도 많습니다.
2	titanic_df['Embarked'].fillna('S')	카테고리형 컬럼은 최빈값(most frequent) 으로 대체하는 것이 일반적입니다. Embarked의 최빈값은 S(644건)이므로 결국 2건을 'S'로 채웁니다.
3	titanic_df.isna().sum()	대체 작업의 검증 단계 입니다. 모든 컬럼이 0으로 나와 결측치가 완전히 제거되었음을 확인합니다. 전처리 후에는 항상 이렇게 검증하는 습관이 중요합니다.

▶ 실행 결과

```
PassengerId    0
Survived        0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          0
Embarked        0
dtype: int64
```

9. apply lambda 식으로 데이터 가공

판다스 내장 함수만으로 처리하기 어려운 **복잡한 데이터 가공**이 필요할 때 **apply** 에 lambda 식을 결합해 사용합니다.

9.1 lambda 식의 기초

```
def get_square(a):
    return a**2

print('3의 제곱은:', get_square(3))
```

줄	코드	상세 설명
1~2	def get_square(a): return a**2	비교를 위한 일반적인 함수 정의 입니다. def 키워드, 함수명, 입력 인자, return 문으로 구성됩니다.
4	print('3의 제곱은:', get_square(3))	3의 제곱인 9가 출력됩니다.

▶ 실행 결과

```
3의 제곱은: 9
```

```
lambda_square = lambda x : x ** 2
print('3의 제곱은:', lambda_square(3))
```

줄	코드	상세 설명
1	lambda_square = lambda x : x ** 2	lambda 식은 함수의 선언과 처리를 한 줄로 압축 한 것입니다. 구조는 lambda 입력인자 : 반환할 표현식이며, 콜론(:) 왼쪽이 입력, 오른쪽이 return 이 생략된 반환값입니다. 함수명 없이 즉석에서 만들어 쓴다고 해서 익명 함수 (anonymous function) 라고도 합니다.
2	print(lambda_square(3))	결과 9로, 위의 def 함수와 완전히 동일 합니다.

▶ 실행 결과

```
3의 제곱은: 9
```

```
a=[1,2,3]
squares = map(lambda x : x**2, a)
list(squares)
```

줄	코드	상세 설명
2	squares = map(lambda x : x**2, a)	lambda는 여러 개의 값을 한 번에 처리할 때 진가를 발휘합니다. map(함수, 반복가능객체) 는 리스트의 각 원소에 함수를 적용합니다. 반환값은 즉시 계산되지 않는 map 객체(지연 평가) 입니다.
3	list(squares)	list() 로 감싸야 실제로 계산되어 [1, 4, 9] 가 출력됩니다. 판다스의 apply 가 바로 이 map과 같은 역할을 Series에 대해 수행합니다.

▶ 실행 결과

```
[1, 4, 9]
```

9.2 apply lambda로 파생 컬럼 만들기

```
titanic_df['Name_len'] = titanic_df['Name'].apply(lambda x : len(x))
titanic_df[['Name', 'Name_len']].head(3)
```

줄	코드	상세 설명
1	titanic_df['Name'].apply(lambda x : len(x))	apply() 는 Series의 각 원소를 하나씩 lambda의 x로 전달 하고, 반환값들을 모아 새 Series를 만듭니다. 여기서 x는 승객 이름 문자열이고 len(x) 는 그 길이입니다. 결과를 Name_len 컬럼으로 추가합니다.
2	titanic_df[['Name', 'Name_len']].head(3)	원본 이름과 계산된 길이를 나란히 확인합니다. 'Braund, Mr. Owen Harris'는 23자입니다.

▶ 실행 결과

```

      Name  Name_len
0  Braund, Mr. Owen Harris      23
1  Cumings, Mrs. John Bradley (Florence Briggs Th...    51
2  Heikkinen, Miss. Laina      22
```

```
titanic_df['Child_Adult'] = titanic_df['Age'].apply(lambda x : 'Child' if x <=15 else 'Adult' )
titanic_df[['Age', 'Child_Adult']].head(8)
```

줄	코드	상세 설명
1	lambda x : 'Child' if x <=15 else 'Adult'	lambda에 if-else 삼항 연산자 를 결합해 조건 분기를 구현합니다. 파이썬 문법상 참일 때의 값이 if보다 앞에 옵니다: [참일때] if [조건] else [거짓일때] . lambda 식에는 return이나 여러 줄 문장을 쓸 수 없으므로 이 형태를 사용해야 합니다.
2	titanic_df[['Age', 'Child_Adult']].head(8)	8건을 출력합니다. 인덱스 5의 Age가 29.699118 인 것은 앞에서 평균값으로 결측치를 대체했기 때문이며, 인덱스 7의 2세는 'Child'로 정확히 분류되었습니다.

▶ 실행 결과

```
Age Child_Adult
0 22.000000 Adult
1 38.000000 Adult
2 26.000000 Adult
3 35.000000 Adult
4 35.000000 Adult
5 29.699118 Adult
6 54.000000 Adult
7 2.000000 Child
```

```
titanic_df['Age_cat'] = titanic_df['Age'].apply(lambda x : 'Child' if x<=15 else ('Adult' if x <= 60 else 'Elderly'))
titanic_df['Age_cat'].value_counts()
```

줄	코드	상세 설명
1	'Child' if x<=15 else ('Adult' if x<=60 else 'Elderly')	else 부분에 또 다른 if-else 를 중첩 하여 3개 이상의 분기를 만듭니다. 판정 순서는 ① 15세 이하면 Child, ② 아니면서 60세 이하면 Adult, ③ 그 외는 Elderly입니다. else if 문법이 없으므로 반드시 괄호로 감싸야 합니다.
3	titanic_df['Age_cat'].value_counts()	분류 결과의 분포를 확인합니다. Adult 786명, Child 83명, Elderly 22명입니다.

▶ 실행 결과

```
Age_cat
Adult      786
Child       83
Elderly     22
Name: count, dtype: int64
```

⚠ 중첩의 한계

분기가 3개만 넘어가도 괄호가 겹쳐 **가독성이 급격히 떨어지고 오류를 찾기 어려워집니다**. 이럴 때는 아래처럼 별도 함수를 정의해 apply에 넘기는 것이 정답입니다.

9.3 별도 함수를 정의해 apply에 적용하기

```
# 나이에 따라 세분화된 분류를 수행하는 함수 생성.
def get_category(age):
```

```
cat = ''
if age <= 5: cat = 'Baby'
elif age <= 12: cat = 'Child'
elif age <= 18: cat = 'Teenager'
elif age <= 25: cat = 'Student'
elif age <= 35: cat = 'Young Adult'
elif age <= 60: cat = 'Adult'
else : cat = 'Elderly'
```

```
return cat
```

```
# lambda 식에 위에서 생성한 get_category( ) 함수를 반환값으로 지정.
# get_category(X)는 입력값으로 'Age' 컬럼 값을 받아서 해당하는 cat 반환
titanic_df['Age_cat'] = titanic_df['Age'].apply(lambda x : get_category(x))
titanic_df[['Age', 'Age_cat']].head()
```

줄	코드	상세 설명
2	def get_category(age):	나이를 입력받아 카테고리 문자열을 반환하는 함수를 정의합니다.
3	cat = ''	반환할 변수를 빈 문자열로 초기화합니다.
4~10	if / elif / else 체인	7단계 분기 를 if-elif로 순차 판정합니다. 조건은 위에서부터 차례로 평가 되므로 5세 이하가 먼저 걸리진 뒤 12세 이하가 판정됩니다. 이 순서 덕분에 각 조건에 하한값을 쓸 필요가 없습니다. lambda 중첩 대비 가독성이 압도적으로 좋습니다 .
12	return cat	결정된 카테고리를 반환합니다.
16	titanic_df['Age'].apply(lambda x : get_category(x))	lambda 안에서 정의한 함수를 호출 합니다. Age 값이 x로 전달되고 get_category의 반환값이 결과가 됩니다. 참고로 apply(get_category) 처럼 함수를 직접 전달 해도 동일하게 동작하며 더 간결합니다.
17	titanic_df[['Age', 'Age_cat']].head()	22세는 Student, 38세는 Adult, 26·35세는 Young Adult로 세분화되었습니다.

▶ 실행 결과

```
   Age  Age_cat
0  22.0   Student
1  38.0     Adult
2  26.0  Young Adult
3  35.0  Young Adult
4  35.0  Young Adult
```

■ 머신러닝 실무 포인트 - 구간화(Binning)

연속형 나이를 카테고리로 변환하는 것을 **구간화(Binning/Discretization)**라고 합니다. 비선형 관계를 모델이 학습하기 쉽게 만들고 이상치 영향을 줄이는 효과가 있습니다. 판다스는 `pd.cut( )` (구간 경계 지정)과 `pd.qcut( )` (동일 개수 분할)이라는 전용 함수도 제공하므로 함께 익혀 두면 좋습니다.

10. 핵심 요약 및 머신러닝 활용 포인트

10.1 이번 장 핵심 API 요약표

구분	API	기능	주의 사항
로딩·확인	<code>pd.read_csv()</code>	CSV → DataFrame	첫 줄이 컬럼명으로 자동 인식
	<code>head(N) / tail(N)</code>	앞뒤 N개 로우	기본값 5
	<code>shape</code>	(행, 열) 튜플	속성이므로 괄호 없음
	<code>info()</code>	요약 정보·결측 현황	로딩 직후 필수 호출
	<code>describe()</code>	기술 통계량	숫자형 컬럼만 대상
집계	<code>value_counts()</code>	값별 건수 분포	기본 <code>dropna=True</code>
	<code>count/mean/sum/max</code>	Aggregation	NaN 자동 제외
변환	<code>df.values</code>	DataFrame → ndarray	컬럼명·인덱스 제외됨
	<code>to_dict('list')</code>	DataFrame → 딕셔너리	'records' 옵션도 유용
삭제	<code>drop(labels, axis=1)</code>	컬럼 삭제	axis 기본값은 0이므로 명시 필수
	<code>drop(..., inplace=True)</code>	원본 직접 변경	반환값 None — 대입 금지
인덱스	<code>df.index</code>	Index 객체 추출	변경 불가(immutable)
	<code>reset_index()</code>	인덱스 재부여	Series → DataFrame 변환 효과
선택	<code>df['컬럼']</code>	단일 컬럼	Series 반환 (리스트면 DataFrame)
	<code>iloc[행, 열]</code>	위치 기반	정수만 허용, 끝 미포함
	<code>loc[행, 열]</code>	명칭 기반	레이블 사용, 끝 포함
	<code>df[조건]</code>	불린 인덱싱	복합 조건은 괄호 + <code>&</code> , <code> </code>
정렬·그룹	<code>sort_values(by=[...])</code>	정렬	ascending 리스트로 개별 지정 가능
	<code>groupby('컬럼')</code>	그룹화	집계 함수를 붙여야 결과 생성
	<code>agg({'컬럼':'함수'})</code>	컬럼별 다른 집계	함수는 문자열 로 전달 권장
결손	<code>isna().sum()</code>	컬럼별 결측 건수	결측 확인의 표준 관용구
	<code>fillna(값)</code>	결측치 대체	결과를 다시 대입 해야 반영
가공	<code>apply(lambda x : ...)</code>	원소별 사용자 정의 처리	분기 3개 이상은 별도 함수로

10.2 사이킷런 학습을 위한 연결 고리

- **데이터 진단 루틴**: `read_csv( )` → `head( )` → `info( )` → `describe( )` → `isna( ).sum( )`. 어떤 데이터를 받든 이 순서로 시작하면 됩니다.
- **피쳐/레이블 분리**: `X = df.drop('Survived', axis=1)`, `y = df['Survived']` 또는 `X = df.iloc[:, :-1]`, `y = df.iloc[:, -1]`.
- **결측치 처리**: 숫자형은 평균/중앙값, 카테고리형은 최빈값이 기본 전략입니다. 사이킷런의 `SimpleImputer` 로도 동일하게 처리할 수 있습니다.

- **피처 엔지니어링** : Family_No(가족 수), Name_len(이름 길이), Age_cat(연령 구간)처럼 **기존 컬럼을 조합·변환한 파생 피처**가 모델 성능을 좌우합니다.
- **groupby 분석** : `df.groupby(['Sex', 'Pclass'])['Survived'].mean()` 처럼 그룹별 생존율을 보면 어떤 피처가 예측에 유용한지 빠르게 파악할 수 있습니다.
- **모델 입력 변환** : 사이킷런은 DataFrame도 직접 받지만, ndarray가 필요하다면 `df.values` 로 변환합니다. object 타입 컬럼은 반드시 인코딩 후 사용해야 합니다.

⚠ 안정화 버전 참고 (pandas 2.3.x 기준)

- `value_counts()` 와 `reset_index()` 의 결과 컬럼명이 1.x와 다릅니다(`count` , 원본 컬럼명 사용).
- `fillna(..., inplace=True)` 를 컬럼 단위로 체이닝하면 경고가 발생합니다 — **결과 재대입 방식**을 사용하세요.
- `agg([max, min])` 처럼 내장 함수를 넘기면 FutureWarning이 발생합니다 — `agg(['max', 'min'])` **문자열 방식**을 사용하세요.
- `df.append()` 는 제거되었습니다 — `pd.concat()` 을 사용하세요.